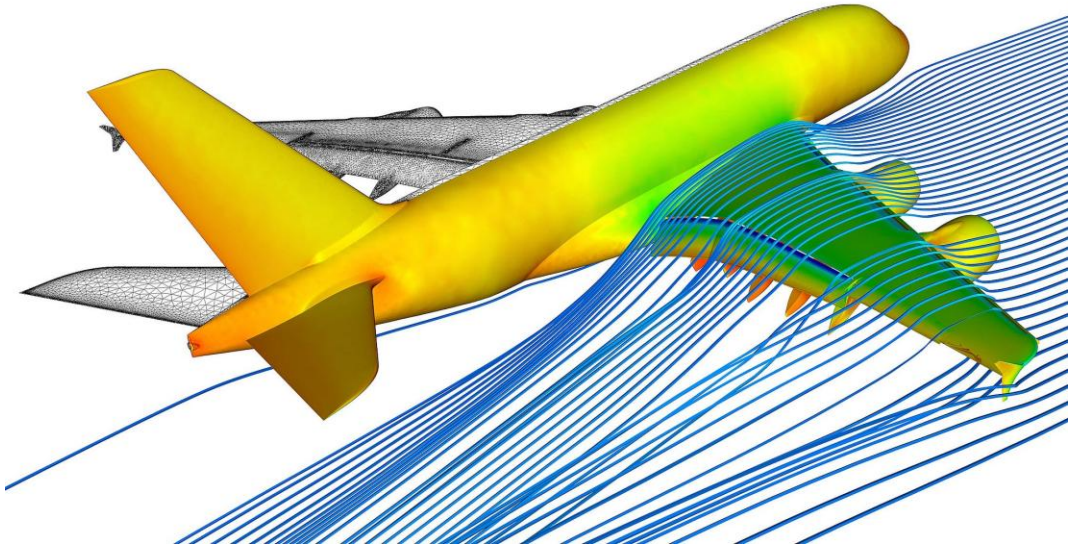
Automating goal-based adaptive mesh refinement for engineering problems

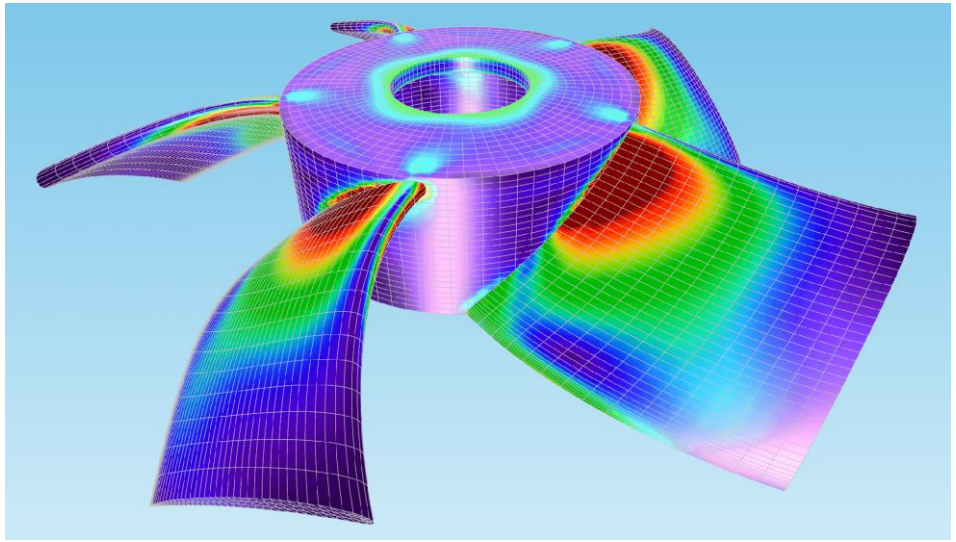
Joe Flood

Motivation: Engineering simulation

- Engineers care about Quantities of Interest **(QoIs)** – not the entire solution field
- Understanding the error in QoIs is important



Lift, drag



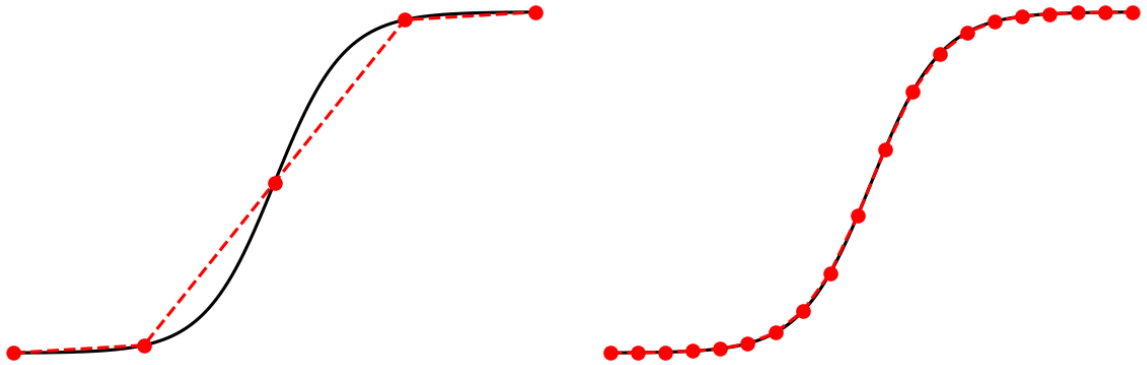
Avg. stress on surface

Background: Finite element methods

PDE		Variational formulation		Finite element formulation		Linear algebra problem		Solution
$\mathcal{L}u = f$	$\longrightarrow$	$\mathrm{a}(u, v) = \langle f, v \rangle$	$\longrightarrow$	$\mathrm{a}(u_h, v_h) = \langle f, v_h \rangle$	$\longrightarrow$	$Au_h = f_h$	$\longrightarrow$	u_h
	Weaken differentiability requirements		Approximate on finite-dimensional function space		Quadrature (assembly)		Solve	

$$\langle a, b \rangle_k := \int_k ab \, dx$$

Accuracy depends on the mesh size (h):



Solution: Goal-based adaptive meshing

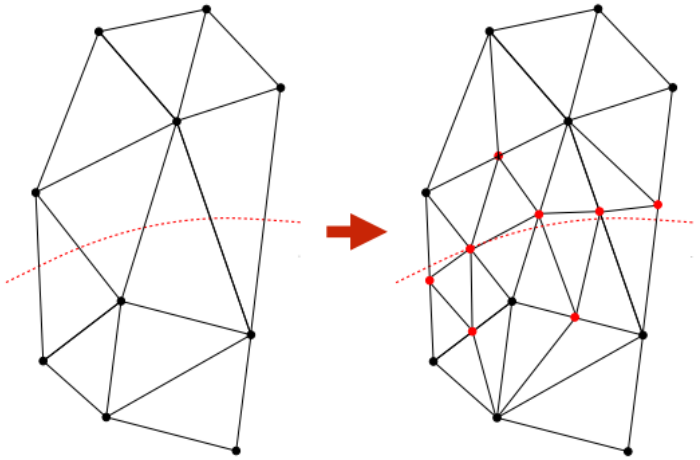
- Define goal functional $J(u) = \int_{x=0} u \, dx$ **QoI**

- Aim 1: Estimate error
- Aim 2: Control error

$$\eta_h := |J(u) - J(u_h)| \leq tolerance$$

- How to control? Adapt mesh!

Use local error indicator η_k



How to estimate η_h? (Global error estimate)

- (Weak) problem: $\boxed{a(u_h, v_h) = F(v_h), \quad \text{for all } v_h \in V_h} \longrightarrow u_h$
- Residual: $r(v_h) := F(v_h) - a(u_h, v_h), \quad$ for any $v_h \in V_h$
- Dual problem: $\boxed{a^*(z_h, v) = J(v), \quad \text{for all } v_h \in V_h} \longrightarrow z_h$
- **Global error:** $\eta_h = |r(z_h)| = |F(z_h) - a(u_h, z_h)|$ (From theorem [1])

Derivation (linear case):

$\eta_h = |J(u) - J(u_h)| = |a^*(z, u - u_h)| = |a(u - u_h, z)| = |F(z) - a(u_h, z)| = r(z) \approx r(z_h)$

How to obtain η_k? (Local error indicator)

- Example: Poisson equation

$$r(z_h) = \langle f, z_h \rangle - \langle \nabla u_h, \nabla z_h \rangle$$

$$= \sum_k \langle f, z \rangle_k - \langle \nabla u_h, \nabla z_h \rangle_k = \sum_k \langle f + \nabla^2 u_h, z_h \rangle_k + \langle -\partial_n u_h, z_h \rangle_{\partial k}$$

$$= \sum_k \langle f + \nabla^2 u_h, z_h \rangle_k + \langle [-\partial_n u_h], z_h \rangle_{\partial k}$$

$$\boxed{\eta_k = |\langle f + \nabla^2 u_h, z_h \rangle_k + \langle [-\partial_n u_h], z_h \rangle_{\partial k}|}$$

- Previously: Had to be derived **manually** for each problem

Automated local error indicators

- A large class of problems have this error indicator form:

$$\eta_k = |\langle R_k(u_h), z_h\rangle_k + \langle [R_{\partial k}(u_h)], z_h\rangle_{\partial k}|$$

- Project onto bubble function b_k:

$$\langle R_k, b_k\rangle_k = r(b_k)$$

- Project onto facet bubble function β_k^s:

$$\langle R_{\partial k}, \beta_k^s\rangle_s = r(\beta_k^s) - \langle R_k, \beta_k^s\rangle_k$$

- Then assemble error indicator:

$$\boxed{\eta_k = |\langle R_k, z\rangle_k + \langle [R_{\partial k}], z\rangle_{\partial k}|}$$

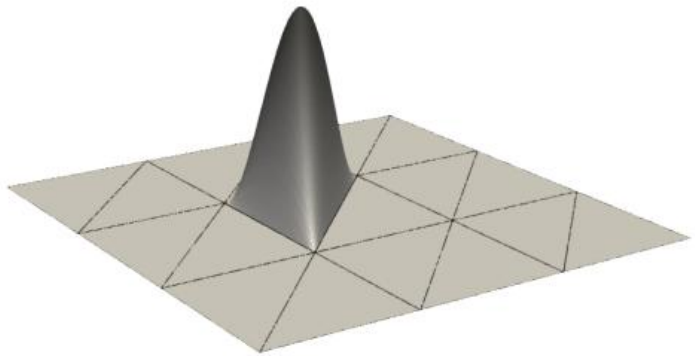


Figure 2.1: Bubble function.

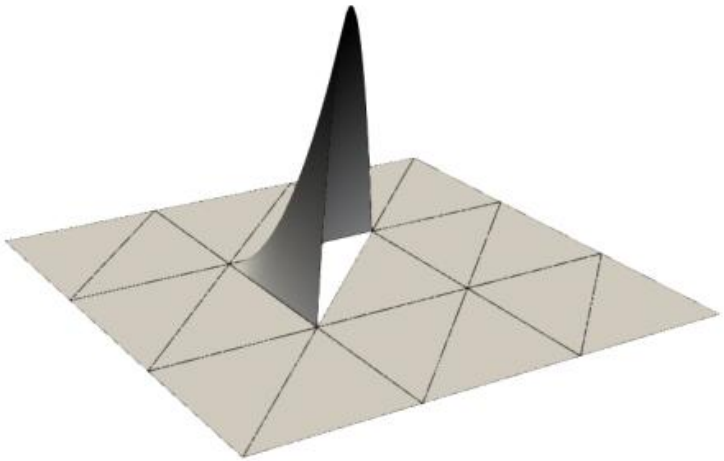
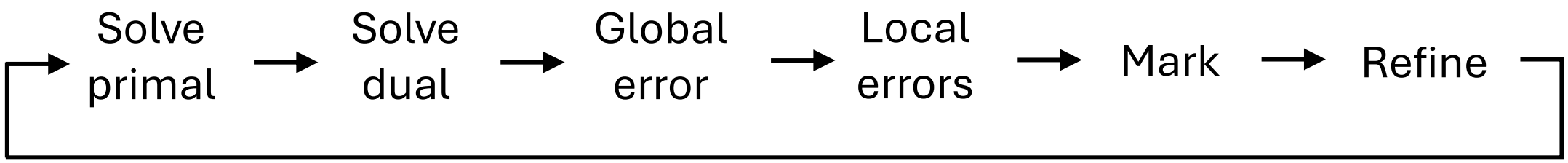


Figure 2.2: Cone function.

Automated algorithm

Given $F(u; v), J(u), tolerance$:

1. Solve primal problem for u_h
2. Solve dual problem for z_h
3. Obtain global error estimate η_h. If $\eta_h \leq tolerance$, break and accept solution
4. Compute local refinement indicators η_k
5. Mark cells for refinement
6. Refine mesh
7. Move problems to new mesh



Implementation (Firedrake)

```
class GoalAdaptiveNonlinearVariationalSolver():
    def __init__(self, problem: NonlinearVariationalProblem, goal_functional,
                 tolerance: float,  solver_parameters: dict, *, primal_solver_parameters = None,
                 dual_solver_parameters = None, exact_solution = None, exact_goal = None,
                 verbose = True, nullspace = None):

    def solve_primal(self):
    def solve_dual(self):
    def compute_etah(self):
    def automatic_error_indicators(self):
    def compute_efficiency(self):
    def mark_cells(self):
    def write_data(self):
    def append_data(self):
    def print(self, *args, **kwargs):
    def solve(self):
    # ... helper functions

def refine_function(expr, self, coefficient_mapping=None):
```

User input

```
mesh = Mesh(unit_square.GenerateMesh(maxh=0.2))

V = FunctionSpace(mesh, "CG", degree=1)
u = Function(V, name="Solution")
v = TestFunction(V)
F = inner(grad(u), grad(v))*dx - inner(f, v)*dx
bcs = [DirichletBC(V, u_boundary, "on_boundary")]

problem = NonlinearVariationalProblem(F, u, bcs)

J = dot(grad(u), FacetNormal(mesh)) * ds
tolerance = 0.0001

solver_parameters = {"max_iterations": 20, "output_dir": "output/poisson_2d"}
sp_dual = {}
sp_primal = {}

GoalAdaptiveNonlinearVariationalSolver(problem, J, tolerance, solver_parameters,
                                       primal_solver_parameters=sp_primal,
                                       dual_solver_parameters=sp_dual).solve()
```

Example: Lift generated by cylinder

Problem:

$$A(u;\varphi) := \nu(\nabla v, \nabla w) + (v \cdot \nabla v, w) - (p, \nabla \cdot w) + (q, \nabla \cdot v), \quad \forall (w,q) \in \hat{V}_h$$

$$J(u) := 500 \int_{\partial \mathcal{B}} \left(\nu \frac{\partial u}{\partial n} - pn \right) \cdot e_2 \, \mathrm{d}s,$$

User input:

```
nu = 0.002
F = (nu * inner(grad(u), grad(v)) * dx + inner(grad(u)*u, v) *dx -
    inner(p, div(v)) * dx + inner(div(u), q) * dx)

bcs = [DirichletBC(T.sub(0), uin, labels["inflow"]),
    DirichletBC(T.sub(0), Constant((0, 0)), [labels["noslip"], labels["cylinder"]])]

# Goal: Lift around cylinder
I = Identity(2)
e2 = as_vector([0.0, 1.0])
M = 500*dot(dot(nu*grad(u) - p*I, n), e2) * ds_cyl

tolerance = 1e-8

problem = NonlinearVariationalProblem(F, u, bcs)
adaptive_problem = GoalAdaptiveNonlinearVariationalSolver(problem,  M, tolerance)
adaptive_problem.solve()
```

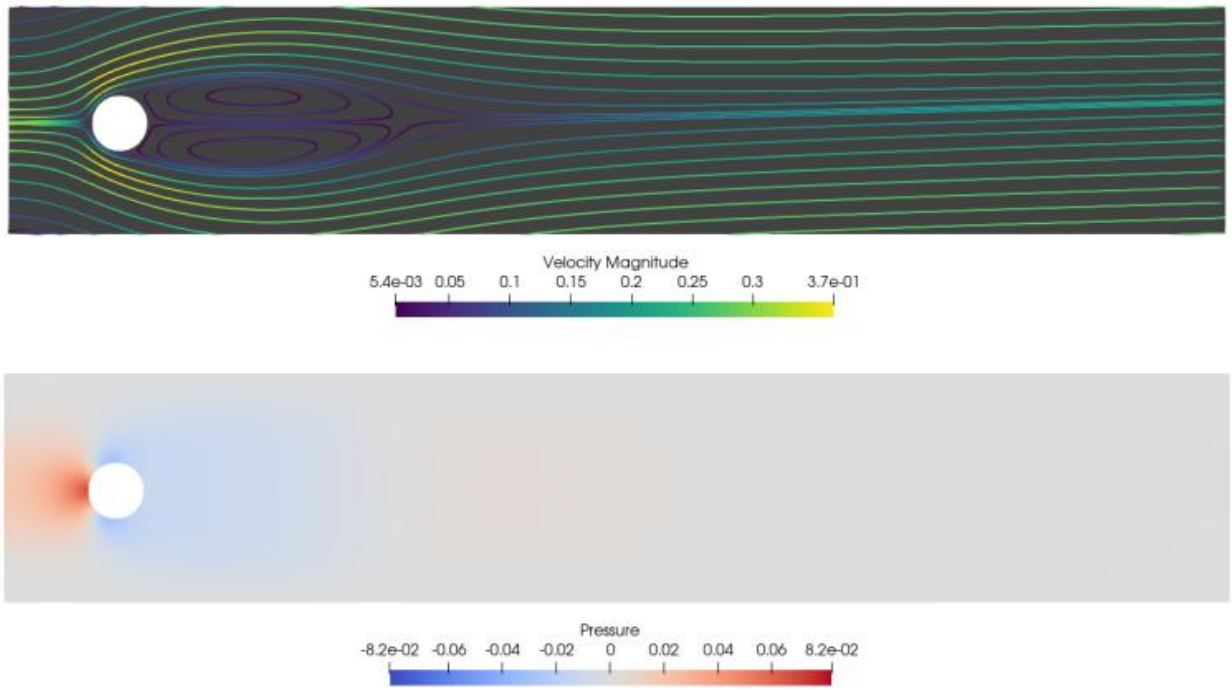
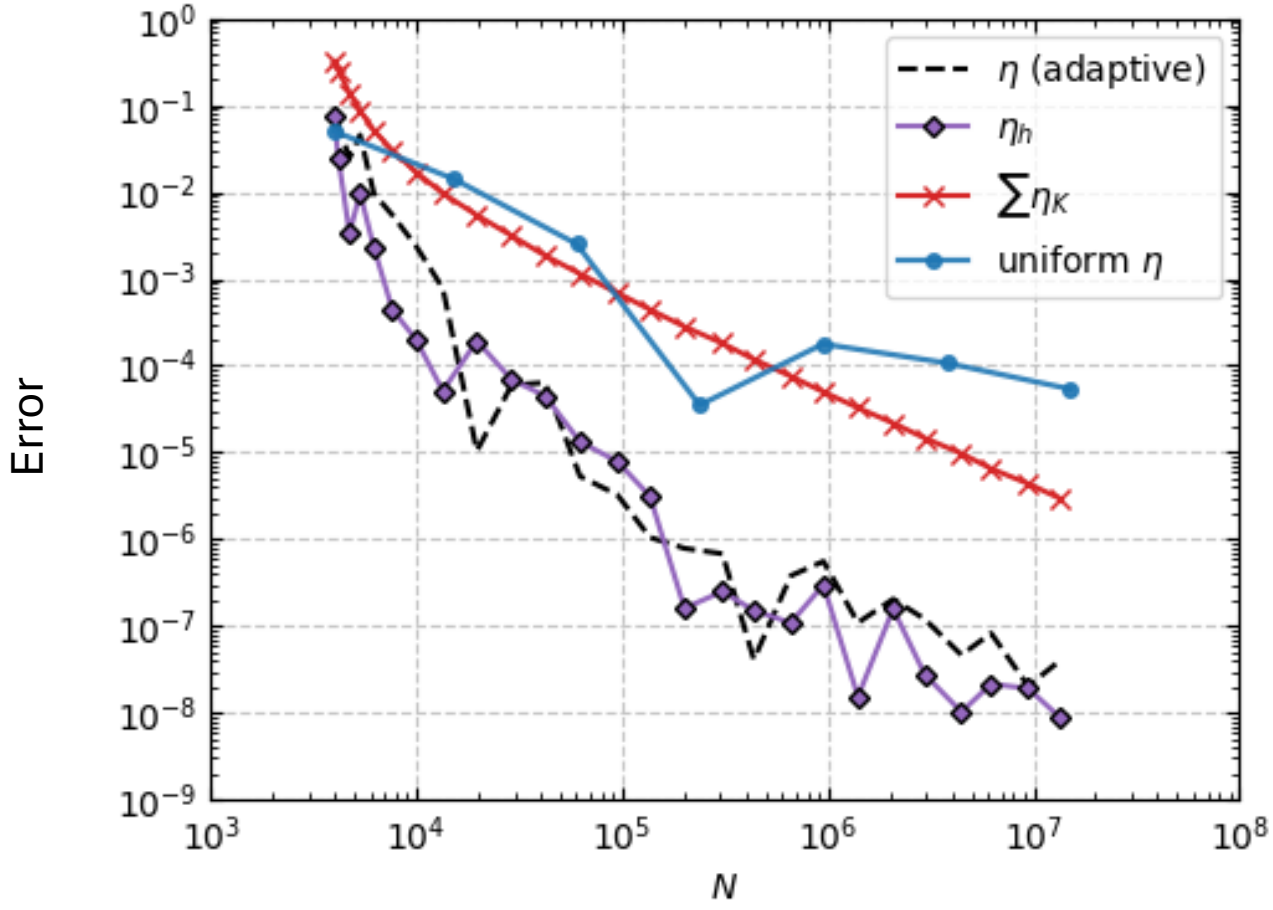


Figure 3.8: Velocity streamlines (top) and pressure field (bottom).

Example: Lift generated by cylinder



Number of degrees of freedom

Example: Lift generated by cylinder

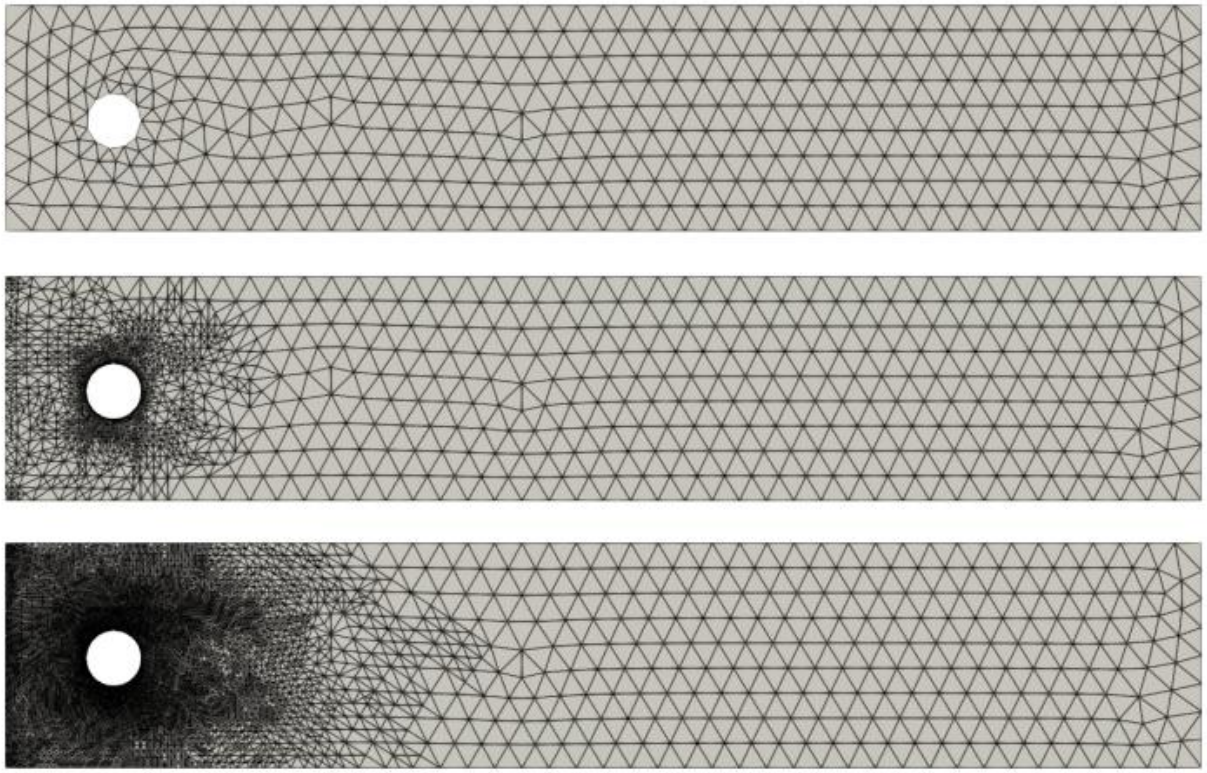


Figure 3.9: Progression of meshes for the Navier-Stokes problem. Top: Initial mesh. Middle: Mesh iteration 9. Bottom: Mesh iteration 18.

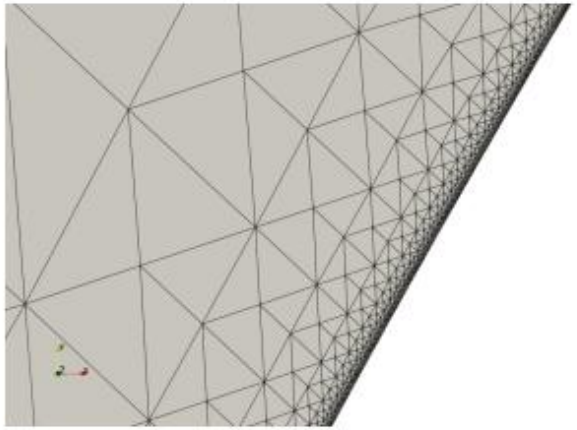


Figure 3.10: Close up of the cylinder boundary on mesh 18.

Extension: Eigenvalue problems

Find $\{u, \lambda\}$ such that:

$a(u, v) = \lambda m(u, v), \qquad$ for all $v \in V$

Given PDE eigenvalue problem, target eigenvalue λ_{target}, and *tolerance:*

- Estimate global error in λ
- Control this error through refinement

- Similar local error structure (but you must consider both primal and adjoint error for non symmetric problems)

$$\eta_K := \left|(R_K, e^*)_K + (R_{\partial K}, e^*)_{\partial K} + (e, R_K^*)_K + (e, R_{\partial K}^*)_{\partial K}\right|$$

Example: Maxwell eigenvalue problem

Problem:

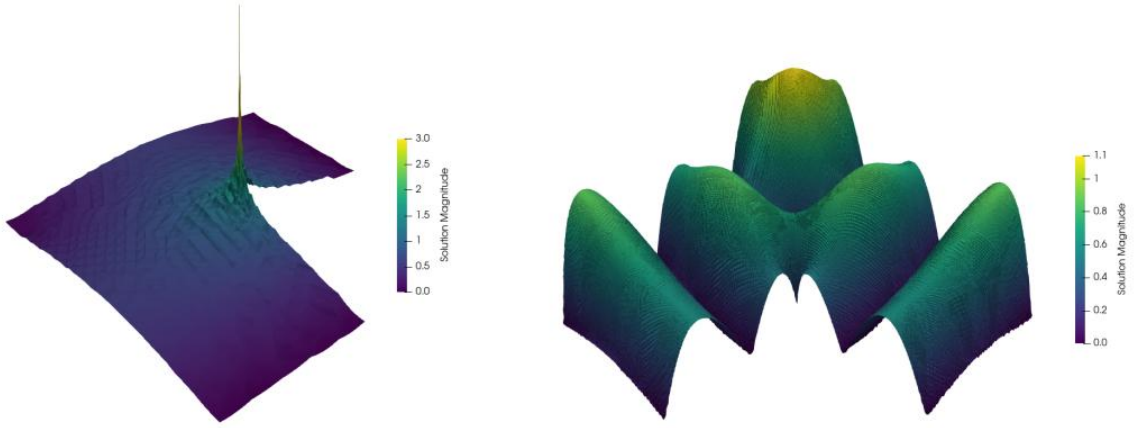
$$a(u,v) = \lambda m(u,v), \quad \forall v \in H_0(\mathrm{curl};\Omega),$$

$$a(u,v) = (\nabla \times u, \nabla \times v)$$

$$m(u,v) = (u,v),$$

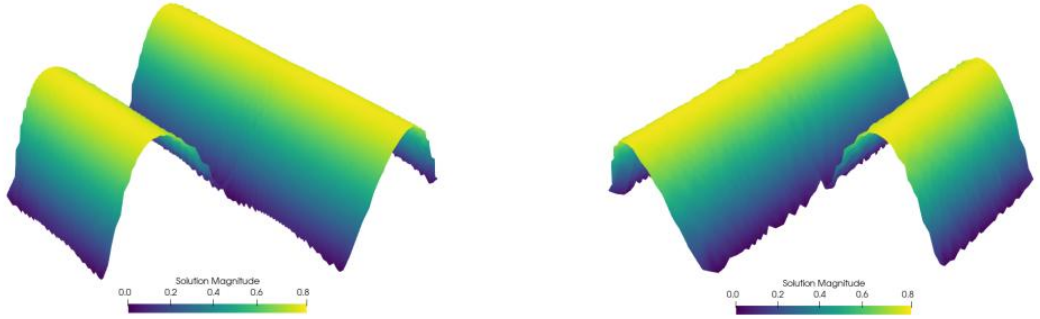
User input:

```
V  = FunctionSpace(mesh, "N1curl", degree=1)
u  = TrialFunction(V); v = TestFunction(V)
A  = inner(curl(u), curl(v)) * dx
M  = inner(u, v) * dx
bcs = [DirichletBC(V, Constant((0.0, 0.0)), "on_boundary")]

target = 12.5723873200
tolerance = 1e-5

problem = LinearEigenproblem(A,M,bcs)
solver = GoalAdaptiveEigenSolver(problem, target, tolerance)
solver.solve()
```



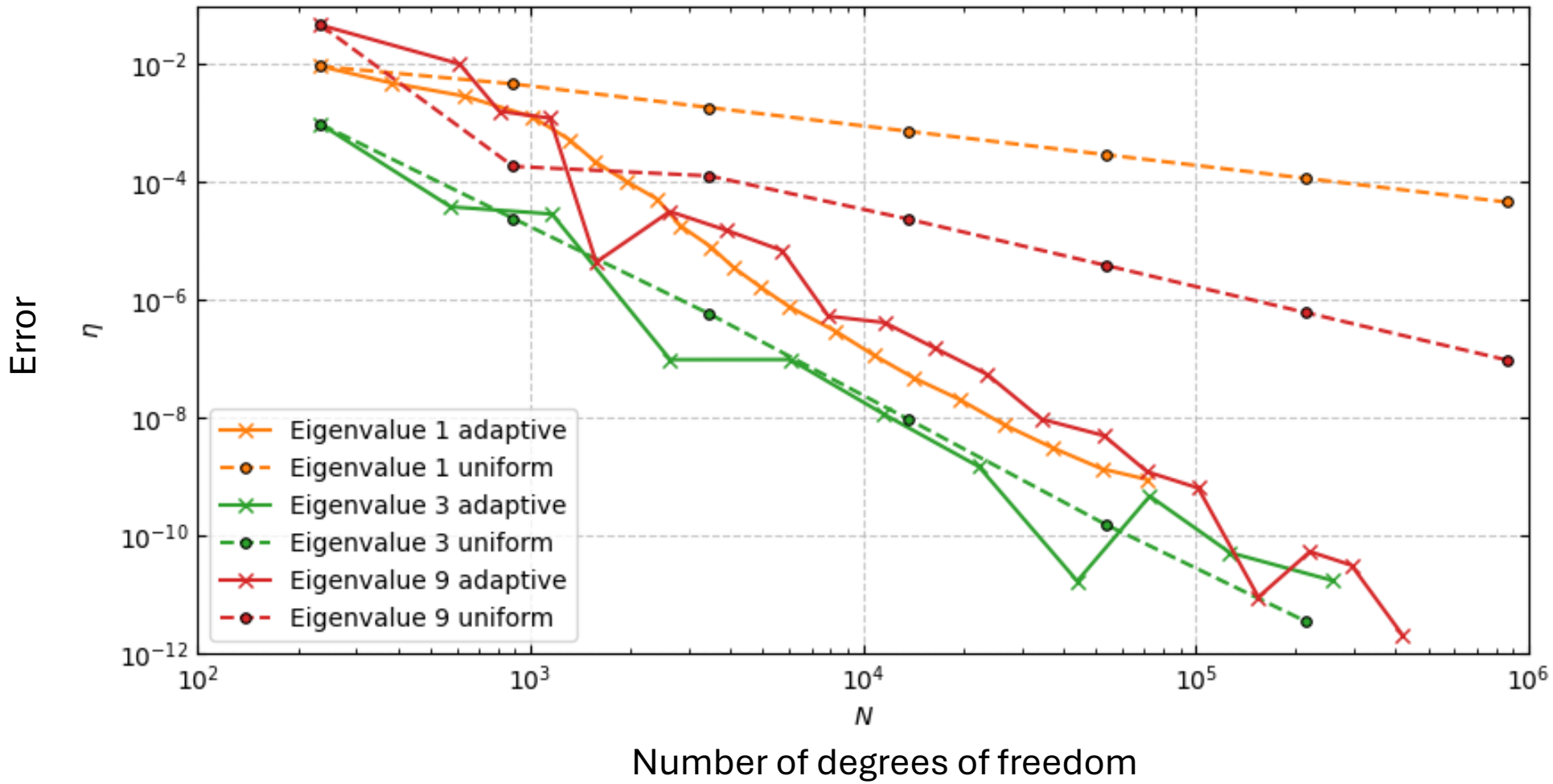
(a) 1st eigenvalue (b) 8th eigenvalue

Figure 4.1: Eigenfunctions corresponding to the 1st and 8th eigenvalues.



(a) Basis 1 (b) Basis 2

Example: Maxwell eigenvalue problem



Questions?

Example: Poisson equation

$$-\nabla^2 u = f$$

$$\int_\Omega -(\nabla^2 u) v \, dx = \int_\Omega f v \, dx$$ For all v in V

$$\int_\Omega \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx - \int_\Gamma \nabla u \, v \, dx$$ For all v in V

$$a(u, v) = F(v)$$ For all v in V